Master SETI

High-Level Synthesis (HLS) on FPGA for Inverse Problems: application to tomography and radioastronomy

Author: Daouda DIAKITE

Supervisor: Nicolas GAC

Université Paris-Saclay, CNRS, CentraleSupélec, Laboratoire des signaux et systèmes, 91190, Gif-sur-Yvette, France.

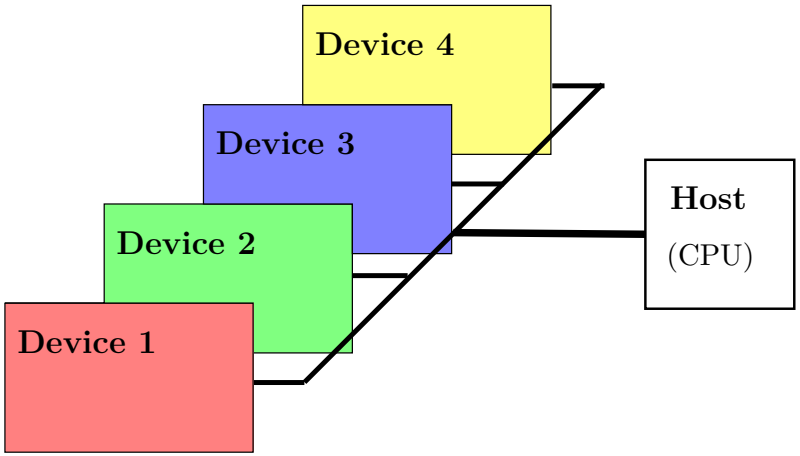
February 02, 2023







FPGA emergency for intensive computing





Advantages of FPGA

A reliable and stable technology well-established in embedded system and prototyping :

- Flexible
- Energy-efficient
- Cycle accurate

Can FPGAs be an alternative to GPUs for intensive computing ?

High-Level Synthesis (HLS)

HLS for embedded systems

- Design fine grained architecture as done in HDL
- Increase productivity for HDL users

High-Level Synthesis (HLS)

HLS for embedded systems

- Design fine grained architecture as done in HDL
- Increase productivity for HDL users

Expectations for HLS intensive computing

- FPGA accessible by a new audience : very HLS (vHLS) tools
- Development time equivalent to GPUs
- Better performance in terms of runtime and energy

High-Level Synthesis (HLS)

HLS for embedded systems

- Design fine grained architecture as done in HDL
- Increase productivity for HDL users

Expectations for HLS intensive computing

- FPGA accessible by a new audience : very HLS (vHLS) tools
- Development time equivalent to GPUs
- Better performance in terms of runtime and energy

FPGA-based HLS performance should be as close as possible to HDL design.

High-Level Synthesis (HLS)

HLS for embedded systems

- Design fine grained architecture as done in HDL
- Increase productivity for HDL users

Expectations for HLS intensive computing

- FPGA accessible by a new audience : very HLS (vHLS) tools
- Development time equivalent to GPUs
- Better performance in terms of runtime and energy

FPGA-based HLS performance should be as close as possible to HDL design.

Applications

- Tomography
- Radioastronomy

Outline

1. FPGA and HLS tools
2. Tomography acceleration : vHLS used as a software developer
3. Tomography acceleration : vHLS used as a hardware designer
4. Conclusion

1 FPGA and HLS tools

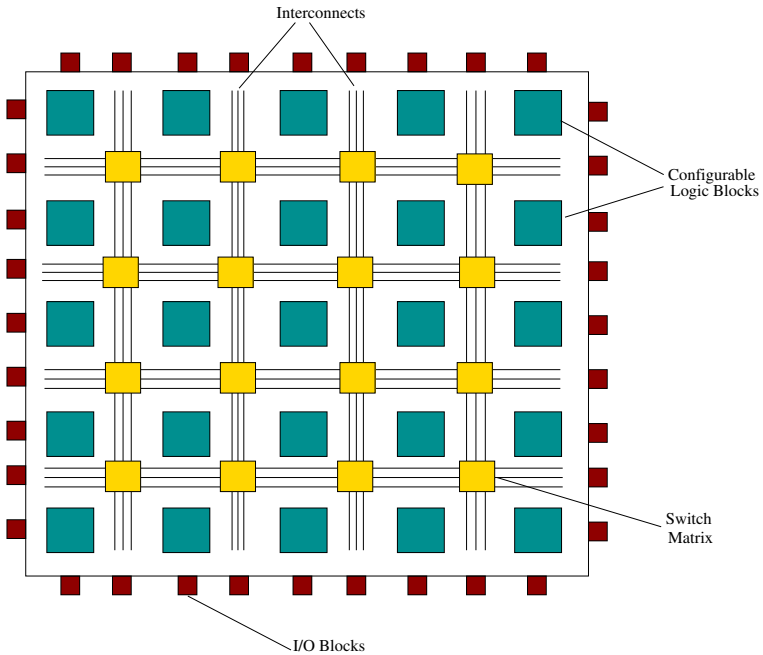
2 Tomography acceleration : vHLS used as a software developer

3 Tomography acceleration : vHLS used as a hardware designer

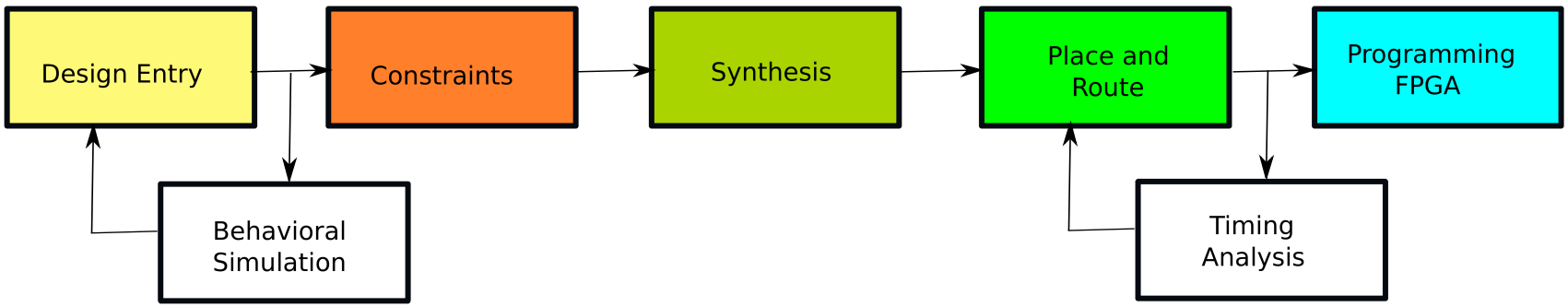
4 Conclusion

Field Programmable Gate Array (FPGA)

- Reconfigurable circuit
- With thousands of memory blocks and DSPs
- Millions of logic resources (FF, LUT, ...)



- FPGA design through HDL language



High-Level Synthesis (HLS) tools

- Academic and Commercial tools
  - The early research since the early 1970s
  - First generation : expectations too high
- Several tools from the literature
  - *GAUT2010, LegUp2011, DWARV2012, ...*
  - *Catapult-C2004, MaxCompiler2010, Intel HLS Compiler2017, ...*

OpenCL HLS (Intel) vs. Vivado HLS (AMD)

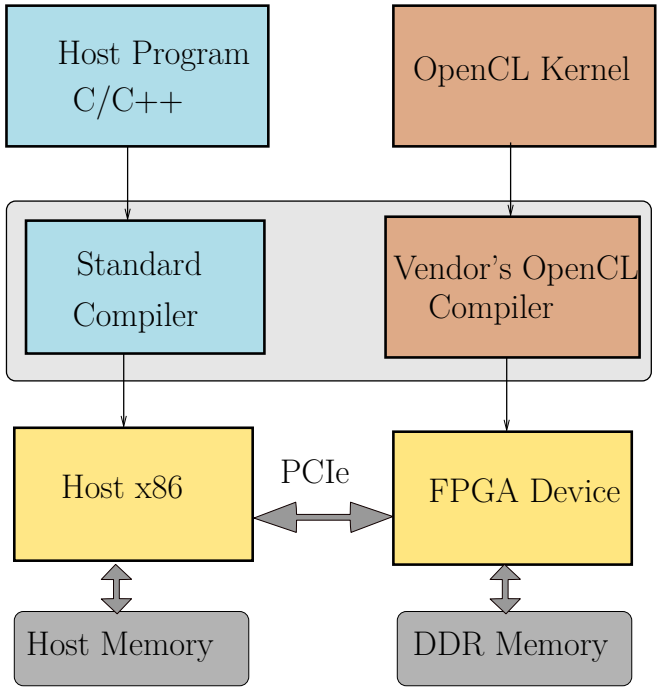
Vivado HLS	**OpenCL SDK**
Xilinx (AMD)	Intel
Design of IP blocks	Kernel development
Traditional design	Host and device system
C/C++	Host API C/C++ and kernel C
For hardware designers	For software developers

- OpenCL can be considered as a very HLS (vHLS) tool.

OpenCL SDK

OpenCL language

- Released to compete with CUDA on GPU
- Host and device platform
- FPGA SDK proposed by vendors

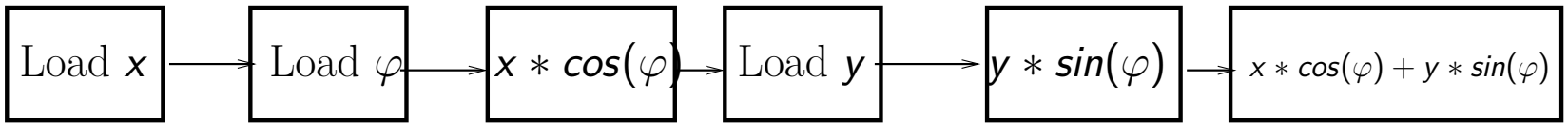


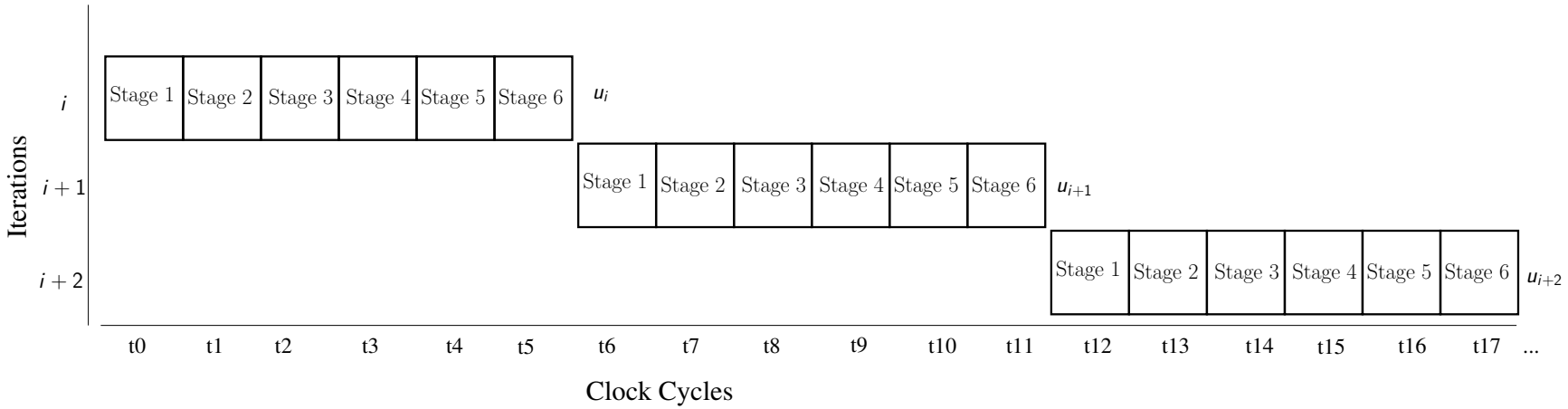
Execution model

- Single Work-Item kernel (SWI)
- NDRange kernel (NDR)

Sequential execution

$$u(\varphi, x, y) = x * cos(\varphi) + y * sin(\varphi)$$

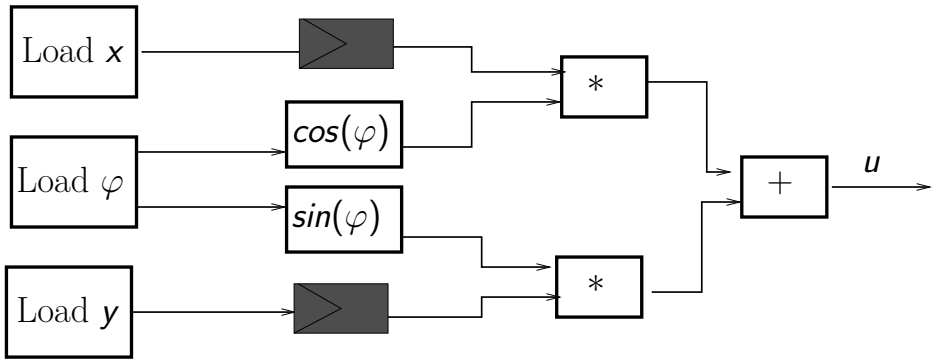


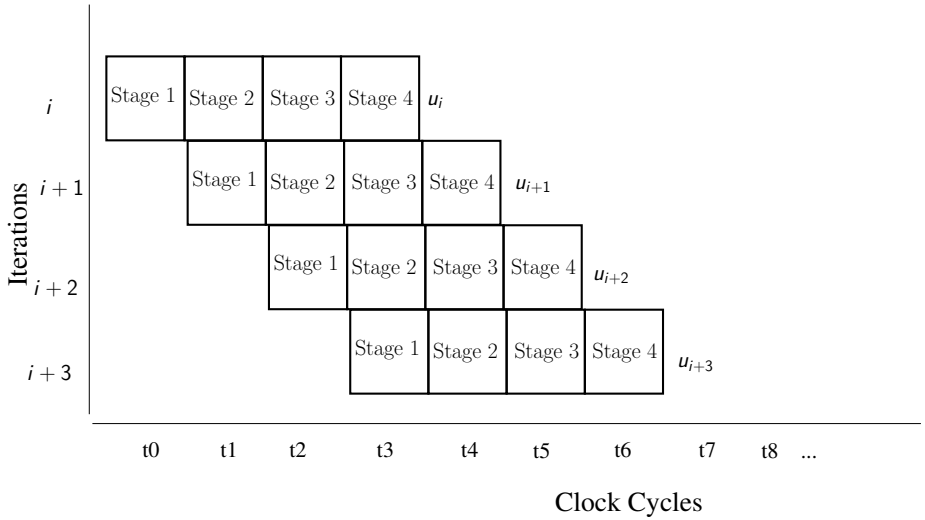


Pipeline execution

- Split the function into micro-operation
- Produce a new result each cycle

$$u(\varphi, x, y) = x * cos(\varphi) + y * sin(\varphi)$$

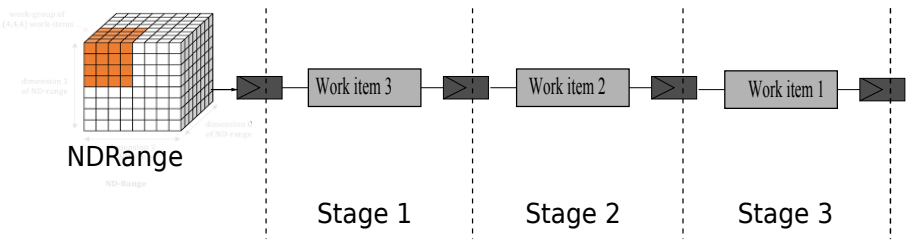




Programming model

NDR kernel

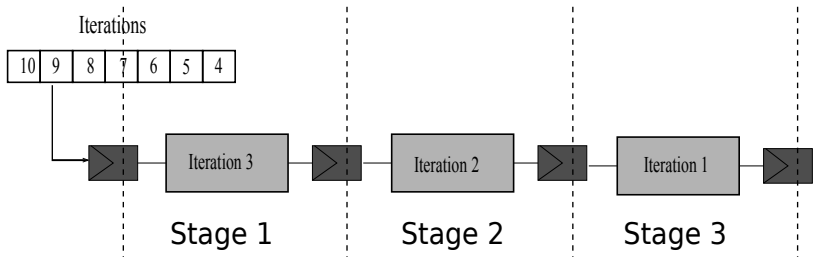
```
__kernel void NDR_kernel()
{
    // get index of the work item
    int index = get_global_id(0);
    // Some computations
}
```



- Each work item handles one piece of data
- Synchronization barrier to resolve dependencies

SWI kernel

```
__kernel void SWI_kernel()
{
    for(int index = 0; index<N; index++){
        // Some computations
    }
}
```



- Dependencies resolved by the compiler
- Initiation interval determined at compile-time

1 FPGA and HLS tools

2 Tomography acceleration : vHLS used as a software developer

3 Tomography acceleration : vHLS used as a hardware designer

4 Conclusion

Tomographic reconstruction

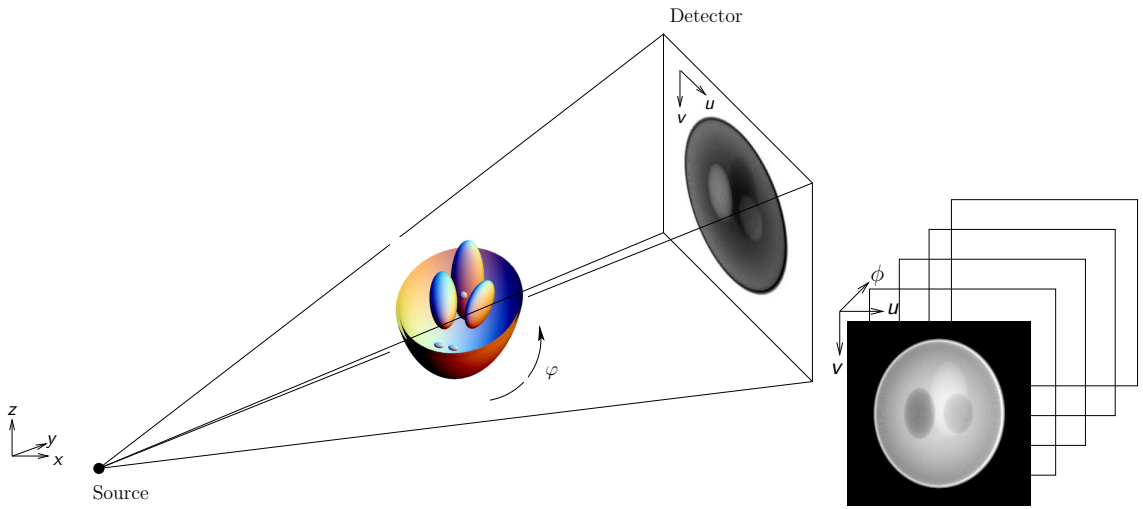


FIGURE – X-RAY CT Projection

Applications

- Non-Destructive testing
- Medical imaging

Iterative reconstruction algorithm

Inverse problem definition

$$g = Hf + \epsilon$$

The object f is obtained by minimizing the cost function :

$$\widehat{f} = \arg\min_{f} J(f)$$

$$J(f) = \frac{1}{2}||g - Hf||^2 + \lambda||Df||^2$$

Gradient descent algorithm

$$f^{(n+1)} = f^{(n)} - \alpha.\nabla J(f^{(n)})$$

$$\nabla J(f^{(n)}) = -2H^t(g - Hf) + \lambda D^t Df$$

$\mathcal{P}/\mathcal{BP}$ pair

On-the-fly computation

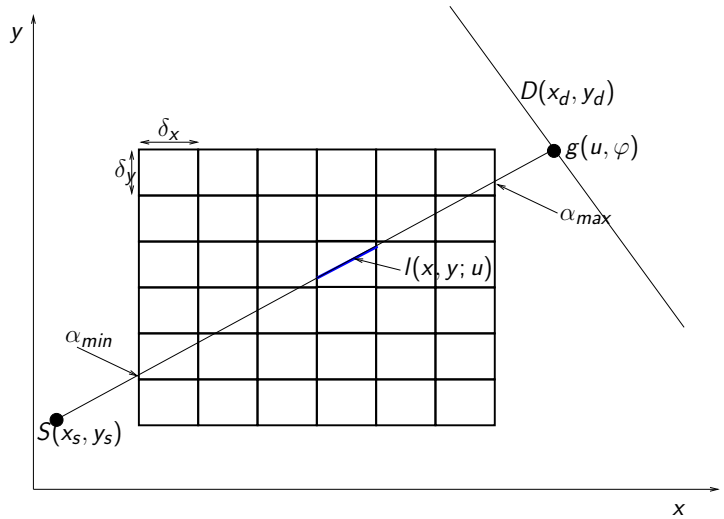
H Forward

H^t Backward

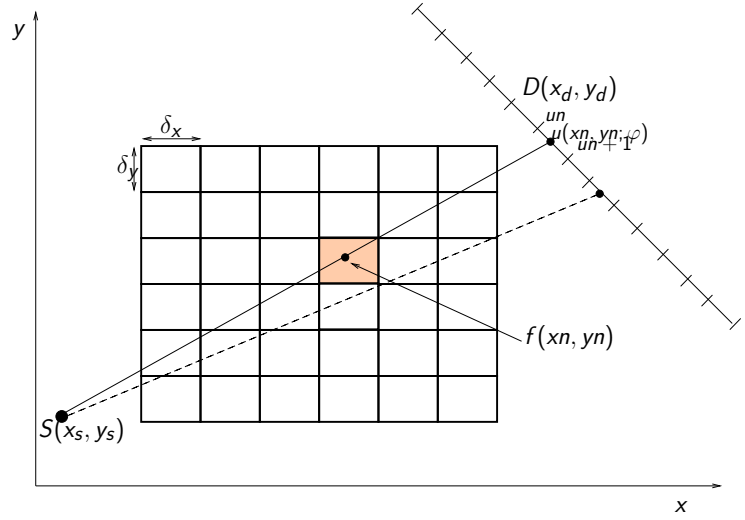
} Huge to be stored $\Rightarrow$ Approach by operators
Forward and backward projections

- Large system matrix size : 2048^3 H $\approx$ 1 Exa Bytes.
- Various methods to model the $\mathcal{P}/\mathcal{BP}$
  - $\mathcal{P}$ Ray-driven / $\mathcal{BP}$ Voxel-driven
  - Dual pair : Distance-driven, Separable footprint

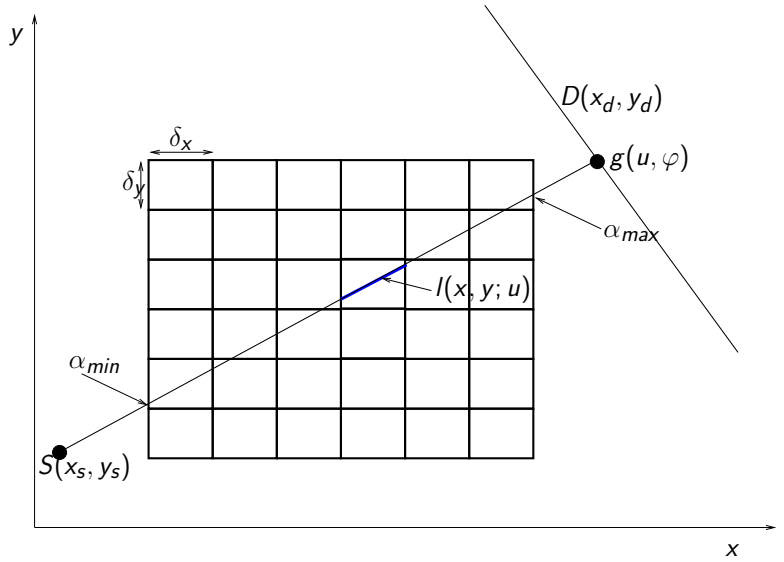
Ray-driven



Voxel-driven



Siddon projector

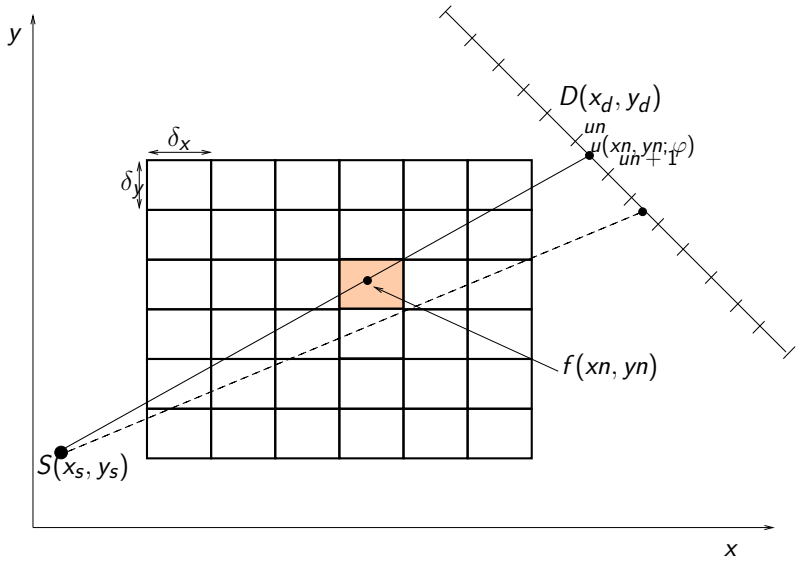


for all φ, un **do**
  Compute L and delta$_\lambda$
  sino$_{ray} \leftarrow 0$
  if $delta_\lambda \geq 0$ **then**
    Compute λ_{min} *and* λ_{max}
    $\lambda \leftarrow \lambda_{min}$
    while $\lambda \leq \lambda_{max}$ **do**
      Compute length l
      Compute (xn, yn)
      $sino_{ray} += l * volume[xn, yn]$
      Update λ
    end while
  end if
  sinogram$[un, \varphi] = sino_{ray}$
end for

Projection algorithm

$$g_{u,\varphi} = \sum_{x,y} l(x, y; u, \varphi) f(x, y)$$

Back-projection algorithm



for all x, y **do**
  $voxel_{sum} \leftarrow 0$
  #pragma unroll N
  for all φ_j **do**
    Compute (u)
    $\text{voxel}_{sum} += projection[u, \varphi_j]$
  end for
  volume[x,y]=voxel_{sum}
end for

3D Back-projector

$$f(x,y) = \int g(u(\varphi,x,y),\varphi).w(\varphi,x,y)^2 d\varphi$$

$$u(\varphi,x,y) = x * cos(\varphi) + y * sin(\varphi)$$

OpenCL design

OpenCL implementations

- SWI design : Fine grained parallelism, unrolling
- NDR design : Coarse grained parallelsim, SIMD

OpenCL design

OpenCL implementations

- SWI design : Fine grained parallelism, unrolling
- NDR design : Coarse grained parallelsim, SIMD

OpenCL optimizations

- Using pragmas and attributes
- Choosing the memory type
- Using cache mechanism
- Applying compiler-assisted optimizations

Design evaluation

Experiment setup

- Volume size 256^3
- Projections data size 256^3
- FPGA Arria 10 device (1150K LEs, 1518 DSPs)

Operator	Kernel	BRAM (%)	Freq. (Mhz)	Stall (%)	Time (s)
Siddon-Cache	SWI	60	188	**87.7**	10.8
	NDR	91	225	**90**	3.8
BP-Cache	SWI	72	150	**71**	**3.5**
	NDR	94	168	**79**	28

- Extensive BRAM block utilization.
- SWI better than NDR

FPGA vs. GPU

Operator	Board	Freq. (Mhz)	Power (Watt)	Time (s)	Energy (mWh)
BP	FPGA Arria 10	150	**23.5**	**3.5**	**22.8**
	GPU A100	1410	**250**	**0.009**	**0.6**

TABLE – Performances comparison (*Diakite et al., 2020*)

Acceleration

- GPUs faster than FPGAs (**2 orders of magnitude**)
- GPUs well-suited for acceleration of regular computation

FPGA vs. GPU

Operator	Board	Freq. (Mhz)	Power (Watt)	Time (s)	Energy (mWh)
BP	FPGA Arria 10	150	**23.5**	**3.5**	**22.8**
	GPU A100	1410	**250**	**0.009**	**0.6**

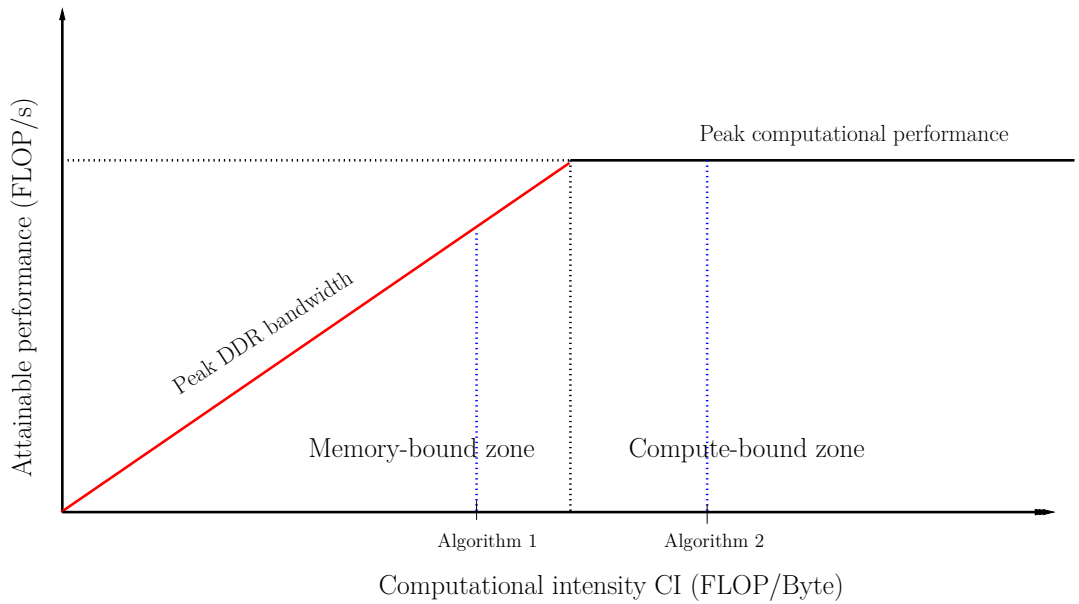
TABLE – Performances comparison (*Diakite et al., 2020*)

Acceleration

- GPUs faster than FPGAs (**2 orders of magnitude**)
- GPUs well-suited for acceleration of regular computation

How about the efficiency of the FPGA pipeline ?

Berkeley Roofline (*Williams et al., 2009*) [1]

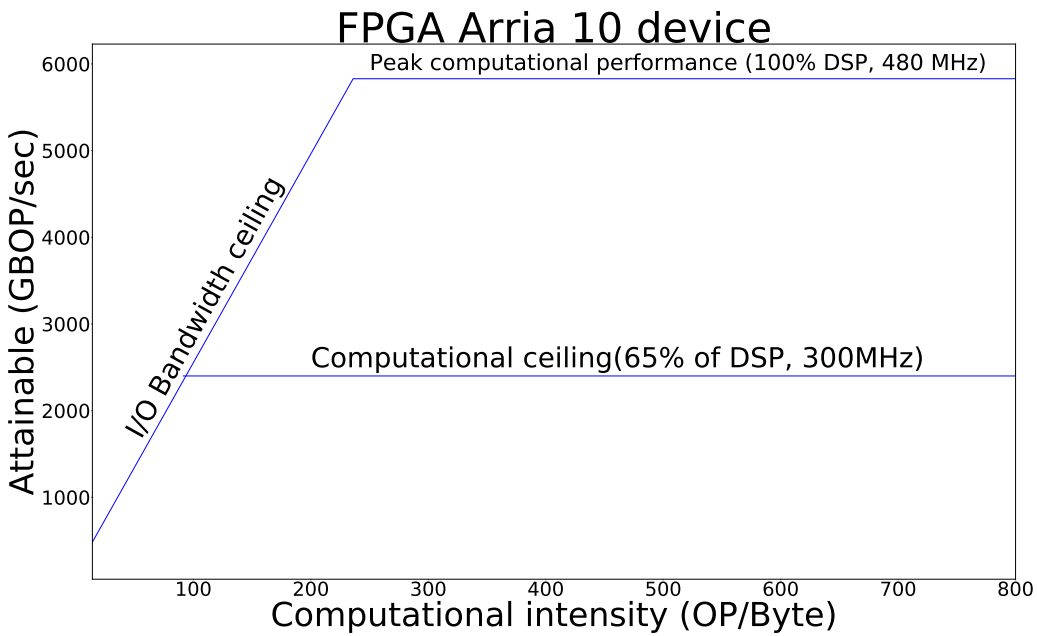


Roofline keys parameters

- The device computational peak performance
- The attainable bandwidth

1. **Williams et al. "Roofline: an insightful visual performance model for multicore architectures." in Communications of the ACM. Apr 2009**.

Roofline for FPGAs (*Silva et al., 2013*) [2]

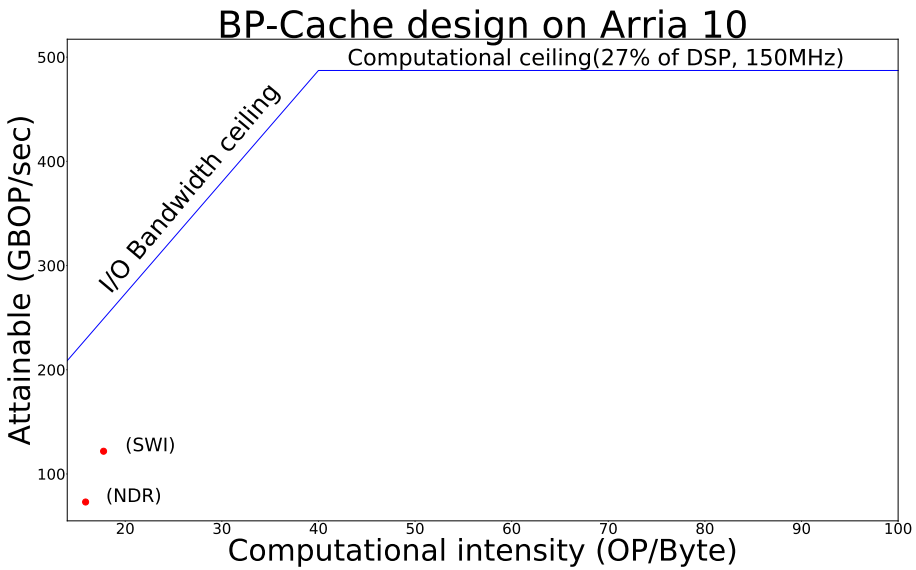


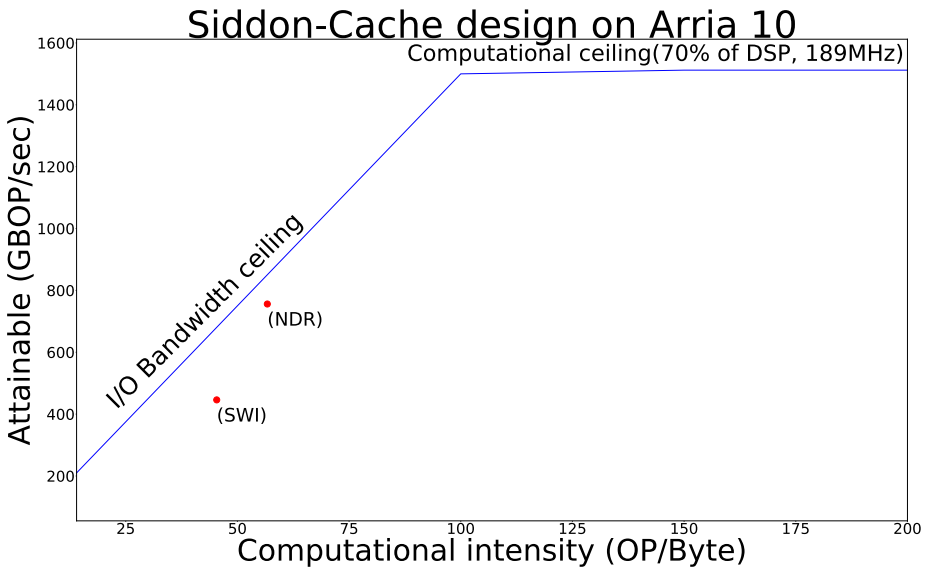
Extention for FPGAs

- Operations
- Computational ceiling
- Bandwidth

2. **Silva et al. "Performance modeling for FPGAs: Extending the roofline model with high-level synthesis tools" in International Journal of Reconfigurable Computing 201**

Roofline model on Arria 10





Main bottlenecks

- Bounded by memory
- Intel automatic cache wasteful (High BRAM usage)
- High memory stall and low occupancy percentage

1 FPGA and HLS tools

2 Tomography acceleration : vHLS used as a software developer

3 Tomography acceleration : vHLS used as a hardware designer

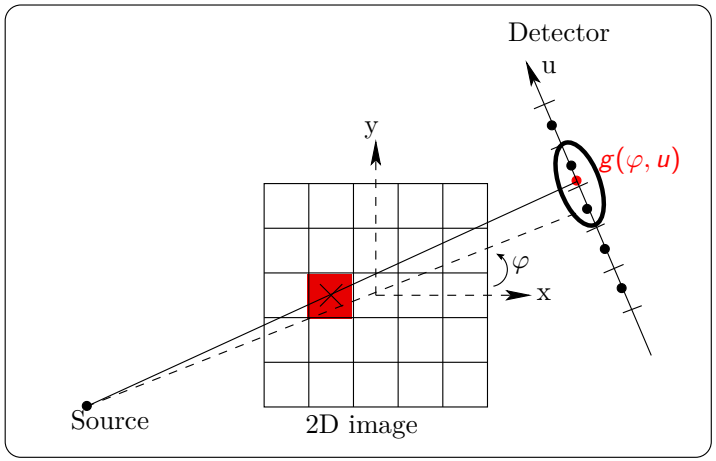
4 Conclusion

Memory datapath for 3D back-projection

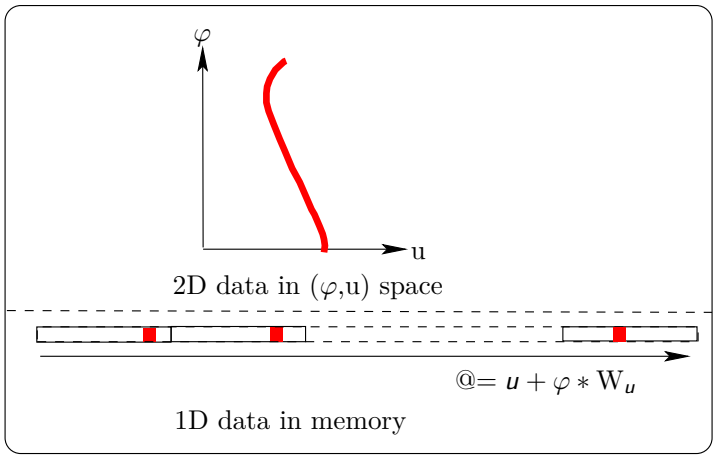
3D Back-projector

$$f(x,y) = \int g(u(\varphi,x,y),\varphi).w(\varphi,x,y)^2 d\varphi$$

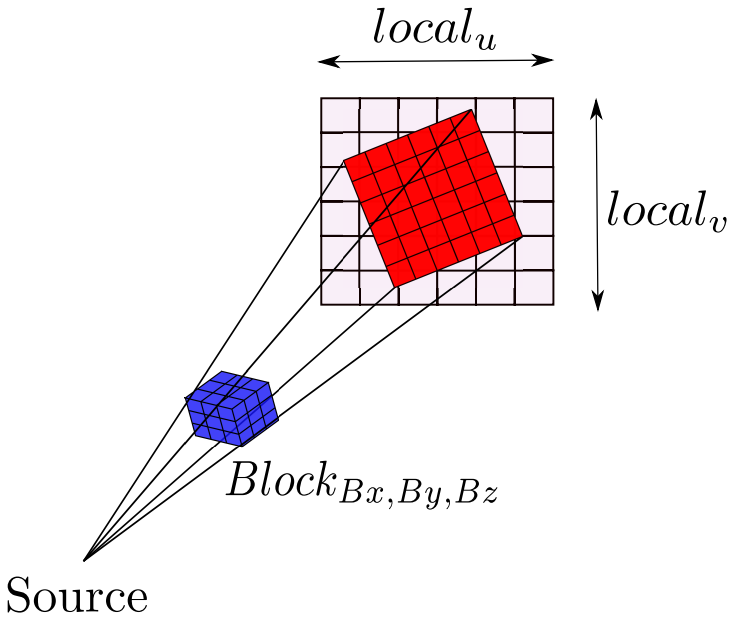
- Voxel by voxel reconstruction results in irregular access.
- High global memory transactions
- Low computational intensity



ACCESS TO PROJECTION DATA $g(\varphi, u)$



Memory access strategy



$$local_u = \sqrt{B_x^2 + B_y^2}$$

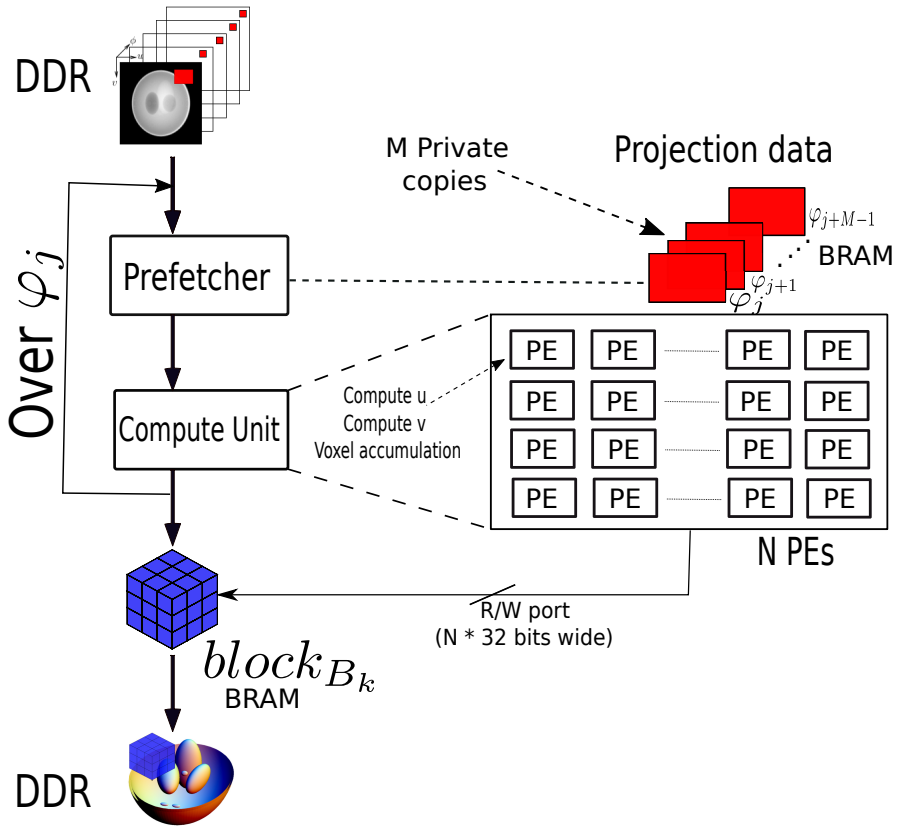
$$local_v = \sqrt{2} * B_z$$

$$Reuse = \frac{B_x * B_y * B_z}{\#Memory\ I/O}$$

Block size and shape decision

- Maximize the data reuse rate
- Optimize the BRAM usage
- Reduce access to global memory

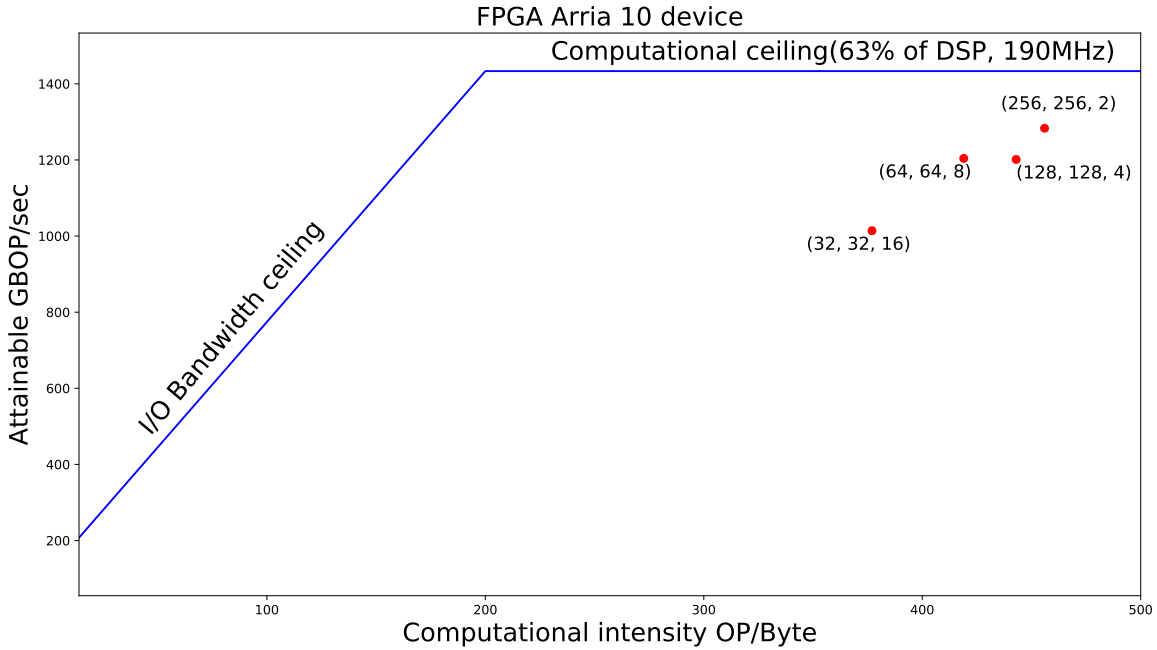
Design architecture on Arria 10



Architecture on FPGA

- Pipeline with N=64 PEs.
- Each PE has a stall-free access to BRAM.

Results on Arria 10



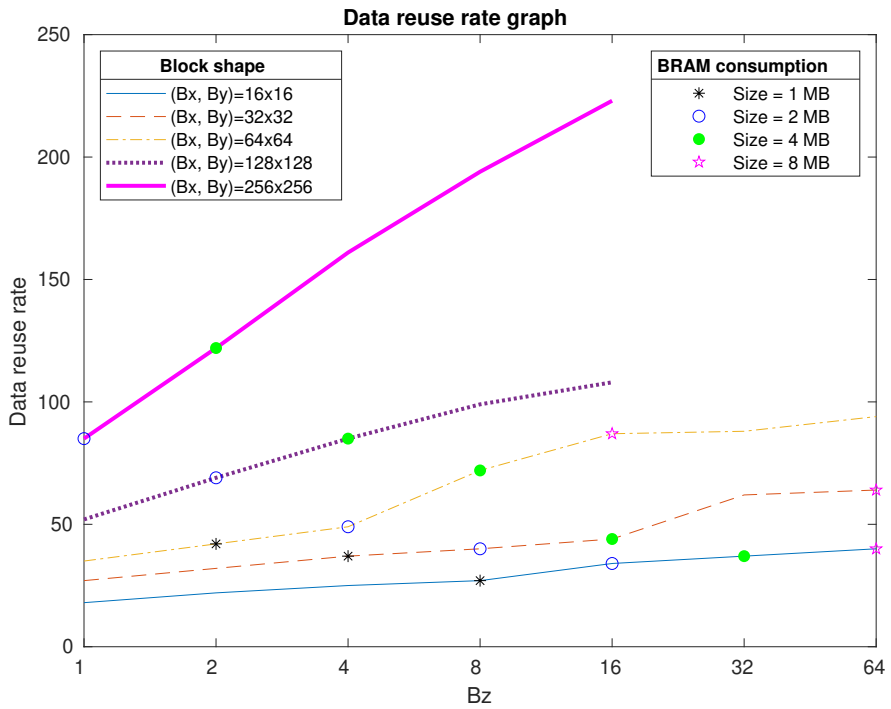
Reference	Platform	Volume	Acc. /voxel	Freq (Mhz)	Stall (%)	Time (s)
BP-cache	Arria 10	256^3	256	150	71	3.62
BP-prefetch $256^2 \times 2$	Arria 10	256^3	256	180	0.06	0.396

TABLE – Performance comparison (*Diakite et al., 2021*)

Data reuse rate

Architecture tuning

$$Data\ reuse = \frac{B_x * B_y * B_z}{\#Memory\ \ access\ \ I/O}$$



Scalability to Stratix 10

Stratix 10 device

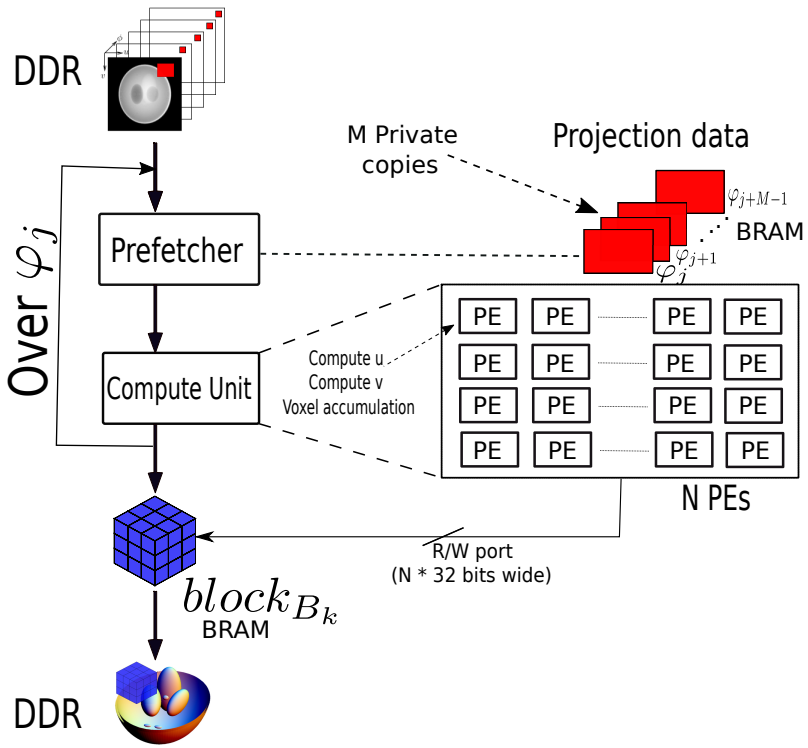
- Stratix 10 device (2800K LEs, 5760 DSPs)
- BP-prefetch Up to 256 PEs on the Stratix 10.

Two strategy

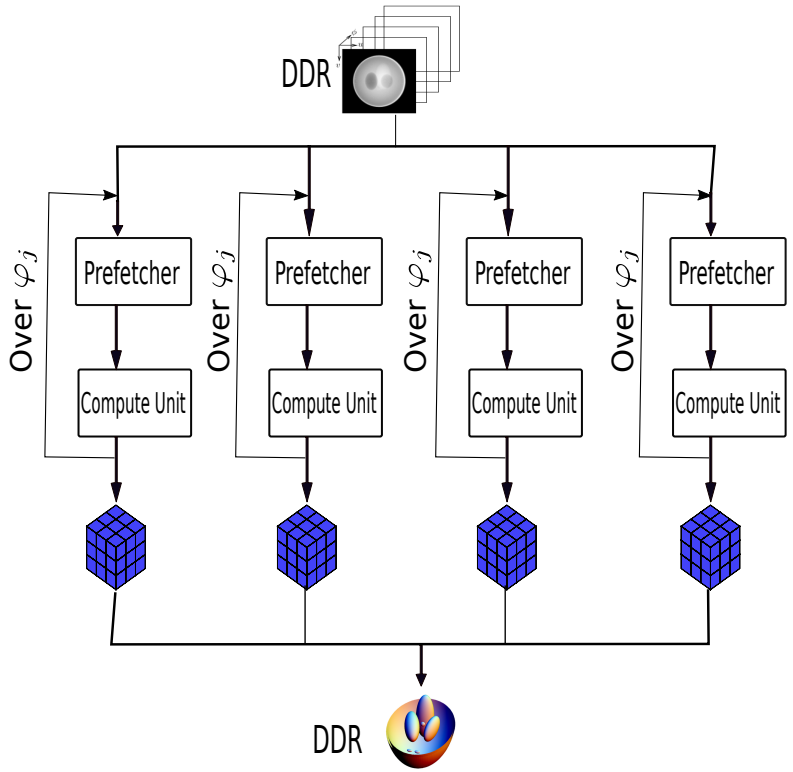
- Single kernel
- Multikernel

Scalability to Stratix 10

Single kernel



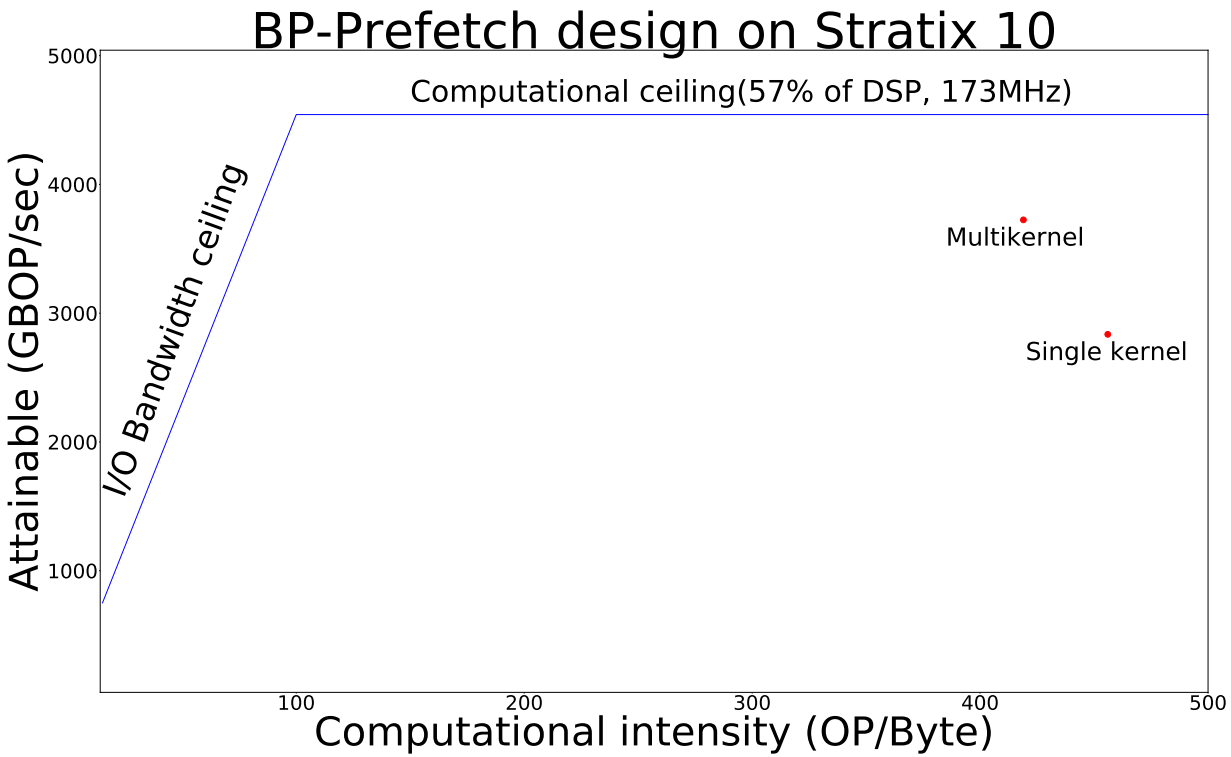
Multikernel



Single vs. multikernel

- Single kernel : $N = 256$ PEs with one kernel
- Multikernel : : $N = 64$ PEs with 4 kernels

Stratix 10 results



Design	Block size	BRAM	DSP	Freq (MHz)	Time (s)
Arria 10	$256^2 \times 2$	1758 (65%)	949 (62%)	180	0.396
Stratix 10 Single kernel	$256^2 \times 2$	5881 (50%)	2845 (49%)	**117**	0.24
Stratix 10 Multikernel	$64^2 \times 8$	3898 (33%)	3282 (57%)	172.5	**0.12**

Comparison with GPU

$$GUPS = \frac{GU}{Time_{kernel}} \quad with \quad GU = \frac{N_{voxel} * N_{acc/voxel}}{1024^3}$$

Comparison with GPU

$$GUPS = \frac{GU}{Time_{kernel}} \quad with \quad GU = \frac{N_{voxel} * N_{acc/voxel}}{1024^3}$$

Evaluation on GPU and FPGA

Device	Power(W)	Time(s)	GUPS	GUPS/Watt
HPC GPU (A100)	250	0.009	424	1.70
Embedded GPU (Jetson TX2)	12,9	0,253	15.8	1.22
Arria 10	14,9	0.42	10.1	0.68
Stratix 10	27	0.12	33.3	1.42

Comparison with GPU

$$GUPS = \frac{GU}{Time_{kernel}} \quad with \quad GU = \frac{N_{voxel} * N_{acc/voxel}}{1024^3}$$

Evaluation on GPU and FPGA

Device	Power(W)	Time(s)	GUPS	GUPS/Watt
HPC GPU (A100)	250	0.009	424	1.70
Embedded GPU (Jetson TX2)	12,9	0,253	15.8	1.22
Arria 10	14,9	0.42	10.1	0.68
Stratix 10	27	0.12	33.3	1.42

FPGA

- 2× faster than embedded GPU
- 10× slower than HPC GPU
- Equivalent energy consumption for Stratix 10 and HPC GPU

Results comparison

Reference	Platform	Volume	Acc. /voxel	Time (s)	GUPS
Choi et al. [3]	Virtex-6	$512^2 \times 372$	831	14.8	5.1
Wen et al. [4]	ZCU102	$1024^2 \times 128$	502	2.10	29.9
Gac et al. [5]	Virtex-4	$128^2 \times 63$	480	0.526	0.88
Our work	Stratix 10	256^3	256	**0.12**	**33.3**

TABLE – Performance comparison of our work and other works

Comparison

- Various datasets
- Ray-driven approach in (*Choi et al.*, *Wen et al.*)
- Use of OpenCL instead of Vivado HLS

3. **Choi et al. "Acceleration of EM-based 3D CT reconstruction using FPGA" in IEEE TBioCAS. 2016.**
4. **Wen et al. "FPGA-accelerated automatic alignment for three-dimensional tomography" in 2020 IEEE 28th FCCM conference.**
5. **Gac et al. "High Speed 3D Tomography on CPU ; GPU ; and FPGA." EURASIP Emb Sys. 2008.**

Efficiency

- We evaluated the pipeline efficiency for each device.
- It represents the number of cycle needed to update one voxel for the pipeline or the CUDA core.

$$\eta = \frac{Time * f_{max} * \#PE}{\#Accumulations}$$

where

$Time$ is the kernel execution time (s)

f_{max} is the operating frequency (Hz)

$\#PE$ is the number of CUDA cores or PEs

$\#Accumulations$ is the total number of voxel updates.

Efficiency

	GPU A100	FPGA Arria 10 BP-Cache	FPGA Arria 10 BP-Prefetch	*(Choi et al., 2016)*
Operations (op)	256^4	256^4	256^4	$512^2 \times 372 \times 831$
Cores	3584 (CUDA cores)	32 (PEs)	64 (PEs)	64 (PEs)
Frequency (MHz)	1410	150	189	100
Time (s)	0.009	5.34	0.42	14.8
cycles/op/core	21.4	**5.97**	**1.18**	**1.17**

HLS vs. HDL

- FPGA VHDL : 1 cycle per PE (*Gac et al., 2008*)
- HLS efficiency : close to the optimal HDL design

Intel ONEAPI

- Programming across different architecture using one single language DPC++
- Easy to use for software and hardware programmers
- Use of FPGA specific pragmas and attributes to optimize the design
- Auto-detect accelerator architecture during application runtime
- Single file for kernel and host applications.

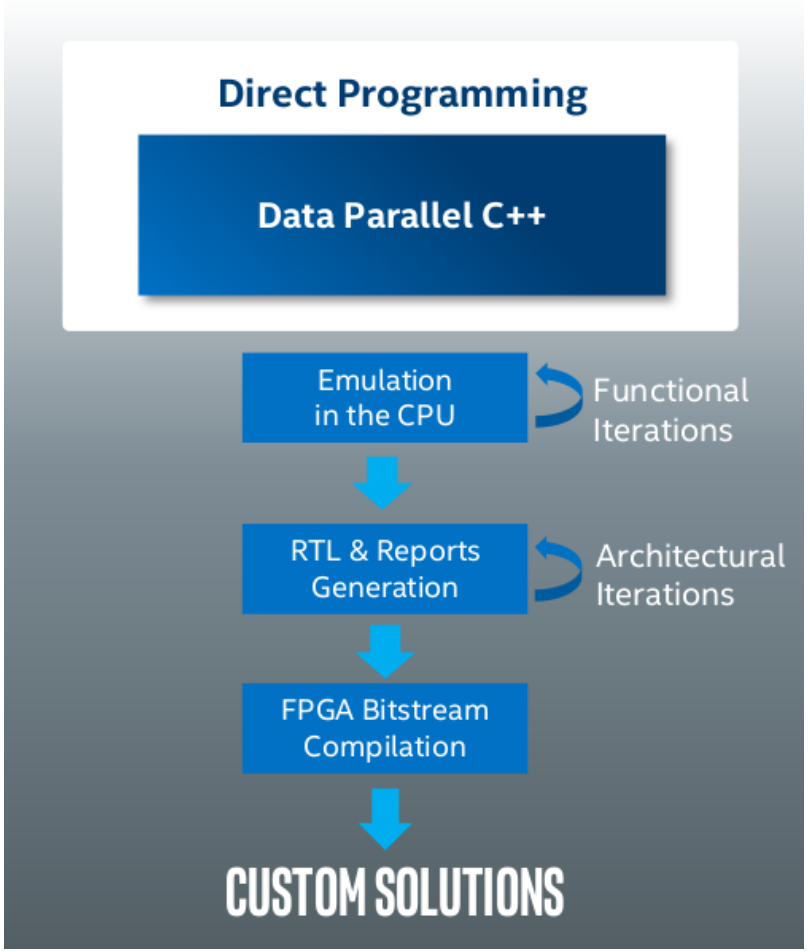


Image source : Intel

Comparison with oneAPI

Comparison between oneAPI and OpenCL on Arria 10

- BP-Cache and the BP-Prefetch designs on Arria 10
- Performance loss for oneAPI
- Number of lines of code reduced compared to OpenCL

Design	Tool	BRAM	DSP	*#PEs*	Freq (MHz)	Time (s)	GUPS
BP-Cache	OpenCL	1958 (72%)	406 (27%)	32	150	3.5	1.14
	OneAPI	2088 (77%)	374 (25%)	32	163	6.43	0.62
BP-Prefetch	OpenCL	1952 (71%)	949 (63%)	64	189	0.396	10.1
	OneAPI	1041 (38%)	878 (58%)	64	229	0.51	7.84

1 FPGA and HLS tools

2 Tomography acceleration : vHLS used as a software developer

3 Tomography acceleration : vHLS used as a hardware designer

4 Conclusion

Conclusion

OpenCL and FPGAs

- Substantial acceleration of computationally intensive algorithms.
- First implementation in a very short time.
- Deep knowledge of FPGA architectures required.

Conclusion

OpenCL and FPGAs

- Substantial acceleration of computationally intensive algorithms.
- First implementation in a very short time.
- Deep knowledge of FPGA architectures required.

FPGA-based HLS

- GPUs are now one order of magnitude faster than FPGA with HLS tools
- FPGAs better than GPUs in terms of worst-case power consumption
- Development time of FPGAs considerably reduced compared to HDL design
- OpenCL design is as close as possible to the HDL design efficiency

Perspectives

FPGA technology

1. Improve FPGA internal architecture
2. Improve HLS design tools

Perspectives

FPGA technology

1. Improve FPGA internal architecture
2. Improve HLS design tools

Algorithm-architecture co-design

1. Apply this methodology to the projection operator
2. Accelerate separable footprint pair on FPGA using HLS
3. Explore multi-FPGA designs, especially for radioastronomy
4. Generate the roofline model automatically through HLS tools

Thank you very much !

Publications

- Journal papers
  - D. Diakite, N. Gac, **X-ray Tomography Reconstruction Accelerated on FPGA through High-Level Synthesis Tools**, *IEEE Transactions on Biomedical Circuits and Systems (Under review)*

- International conferences
  - D. Diakite, N. Gac, M. Martelli, **OpenCL FPGA Optimization guided by memory accesses and roofline model analysis applied to tomography acceleration**, *31st International Conference on Field- Programmable Logic and Applications (FPL), Aug 2021, Dresden, Germany*
  - D. Diakite, M. Martelli, N. Gac, **An OpenCL pipeline implementation on Intel FPGA for 3D backprojection**, *6th International Conference on Image Formation in X-Ray Computed Tomography, Aug 2020, Regensburg, Germany*

- National conferences
  - D. Diakite, N. Gac, **FPGA implementation of 2D Convolution using OneAPI and OpenCL**, *16ème Colloque National du GDR SOC2 2022, Strasbourg, France*
  - D. Diakite, N. Gac, **Premiers résultats de comparaison des outils oneAPI et OpenCL pour la convolution 2D sur FPGA**, *28ème Colloque Francophone de Traitement du Signal et des Images GRETSI 2022, Nancy, France*
  - D. Diakite, N. Gac, **Memory prefetching for tomography acceleration on FPGAs through HLS tools**, *15ème Colloque National du GDR SOC2 2021, Rennes, France*